

**БЮДЖЕТНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
ХАНТЫ-МАНСКИЙСКОГО АВТОНОМНОГО ОКРУГА – ЮГРЫ
«СУРГУТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»**

ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ

Кафедра прикладной математики

КУРСОВАЯ РАБОТА

по дисциплине «Численные методы»

**Метод конечных разностей решения линейной
двухточечной краевой задачи для ОДУ второго порядка**

Выполнил

Студент гр. 601-__ _____ / ФИО.
(подпись)

Проверил

_____ (оценка)
К.ф.-м.н, доцент _____ / Назин А.Г.
(подпись)

Сургут 2026

Содержание

Введение	3
1 Постановка краевой задачи с различными граничными условиями	5
2 Условия разрешимости и единственности	6
3 Вывод разностных формул	7
3.1 Интегро-интерполяционный метод (метод баланса)	7
3.2 Аппроксимация интегралов и потоков	8
3.3 Аппроксимация граничных условий методом фиктивных узлов .	9
3.3.1 Левая граница: условие Неймана	10
3.3.2 Левая граница: условие Робина	11
3.3.3 Правая граница: условие Неймана	12
3.3.4 Правая граница: условие Робина	12
3.4 Условия Дирихле	13
3.5 Итоговая система	13
4 Метод прогонки решения трёхдиагональной СЛАУ	14
4.1 Алгоритм метода прогонки	14
5 Реализация метода конечных разностей для задачи с граничными условиями первого рода (Дирихле)	16
5.1 Результаты сгущения сетки	16
5.2 График численного и точного решений	17
5.3 График оценки погрешности	18
6 Реализация метода конечных разностей для задачи с граничными условиями второго рода (Неймана)	22
6.1 Результаты сгущения сетки	22
6.2 График численного и точного решений	23
6.3 График оценки погрешности	24

7	Реализация метода конечных разностей для задачи с граничными условиями третьего рода (Робина)	26
7.1	Результаты сгущения сетки	26
7.2	График численного и точного решений	27
7.3	График оценки погрешности	28
	Заключение	30
	Список литературы	31
	Приложение	32

Введение

Многие физические, технические и экономические процессы описываются дифференциальными уравнениями. Особый класс составляют **двухточечные краевые задачи** для обыкновенных дифференциальных уравнений (ОДУ) второго порядка. Такие задачи возникают, когда известны условия не в одной точке, а на двух концах некоторого отрезка — отсюда название «двухточечные».

Математическая постановка задачи, рассматриваемой в данной работе, имеет вид:

$$-(k(x)u')' + q(x)u = f(x), \quad a < x < b,$$

где $u(x)$ — искомая функция, $k(x)$, $q(x)$, $f(x)$ — заданные непрерывные функции, причём $k(x) > 0$ на всём отрезке $[a, b]$. Для единственности решения на границах отрезка $[a, b]$ задаются дополнительные условия — первого, второго или третьего рода.

Аналитическое решение такой задачи удаётся найти лишь для простейших случаев. Для реальных задач с переменными коэффициентами приходится использовать **численные методы**. Один из самых распространённых — **метод конечных разностей** (МКР). Суть метода заключается в замене непрерывной области $[a, b]$ набором отдельных точек (узлов), а производных — приближёнными формулами через значения функции в этих точках. В результате дифференциальное уравнение превращается в систему линейных алгебраических уравнений (СЛАУ), которую можно решить на компьютере.

Цель работы: программно реализовать метод конечных разностей для решения линейной двухточечной краевой задачи для ОДУ второго порядка с переменными коэффициентами.

Для достижения поставленной цели решаются следующие **задачи**:

1. Описать постановку линейной двухточечной краевой задачи для ОДУ второго порядка с разными граничными условиями.
2. Указать условия разрешимости краевой задачи.
3. Изложить метод конечных разностей для решения этой задачи.
4. Реализовать в среде Octave алгоритм решения краевой задачи методом

конечных разностей для одномерного стационарного уравнения теплопроводности

$$-\frac{d}{dx} \left(k(x) \frac{du}{dx} \right) + q(x)u = f(x), \quad a < x < b,$$

где $k(x) \neq \text{const}$, $q(x) \neq \text{const}$, $f(x) \neq \text{const}$ — непрерывные функции. Требуемая точность — 10^{-4} .

5. Оценить погрешность численного решения по правилу Рунге.
6. Провести вычислительные эксперименты с различными краевыми условиями.
7. Построить графики полученного решения и график погрешности.

1 Постановка краевой задачи с различными граничными условиями

Рассматривается линейная двухточечная краевая задача для ОДУ второго порядка:

$$-\frac{d}{dx} \left(k(x) \frac{du}{dx} \right) + q(x)u = f(x), \quad a < x < b, \quad (1.1)$$

где $u(x)$ — искомая функция, $k(x)$, $q(x)$, $f(x)$ — заданные непрерывные на $[a, b]$ функции, причём $k(x) > 0$.

Для однозначного определения решения на концах отрезка задаются граничные условия трёх возможных типов:

- **Первый род (Дирихле):** фиксированы значения функции

$$u(a) = A, \quad u(b) = B. \quad (1.2)$$

- **Второй род (Неймана):** фиксированы значения производной

$$u'(a) = \gamma, \quad u'(b) = \delta. \quad (1.3)$$

- **Третий род (Робина):** задана линейная комбинация функции и производной

$$\alpha_0 u(a) + \alpha_1 u'(a) = \gamma_a, \quad \beta_0 u(b) + \beta_1 u'(b) = \gamma_b, \quad (1.4)$$

где $\alpha_0 \neq 0$, $\alpha_1 \neq 0$, $\beta_0 \neq 0$, $\beta_1 \neq 0$.

В дальнейшем для всех трёх типов граничных условий применяется одинаковый алгоритм дискретизации методом конечных разностей.

2 Условия разрешимости и единственности

Для каждой из рассматриваемых краевых задач укажем условия, гарантирующие существование и единственность решения.

Задача Дирихле. Согласно теореме 15.1 из [1], решение краевой задачи с условиями первого рода существует и единственно при выполнении условий:

$$k(x) \geq k_0 > 0, \quad q(x) \geq 0, \quad k \in C^1[a, b], \quad q, f \in C[a, b],$$

где $k_0 > 0$ — некоторая постоянная (нижняя граница функции $k(x)$ на $[a, b]$), которая выбирается исходя из минимального значения $k(x)$ на отрезке и гарантирует, что $k(x)$ отделена от нуля. В тестовом примере $k(x) = x^2 + 1 \geq 1 > 0$, $q(x) = x + 1 \geq 1 > 0$, все функции непрерывны. Условия теоремы выполнены, решение существует и единственно.

Задачи Неймана и Робина. Для задач с граничными условиями второго и третьего рода существование и единственность решения устанавливаются с помощью **теоремы 3.2.2** из [2]. Теорема утверждает: если однородная краевая задача

$$-\frac{d}{dx} \left(k(x) \frac{du}{dx} \right) + q(x)u = 0, \quad a < x < b,$$

с однородными граничными условиями (для Неймана: $u'(a) = 0$, $u'(b) = 0$; для Робина: $\alpha_0 u(a) + \alpha_1 u'(a) = 0$, $\beta_0 u(b) + \beta_1 u'(b) = 0$) имеет только нулевое решение $u \equiv 0$, то решение исходной неоднородной задачи существует и единственно.

Проверка условия теоремы (доказательство того, что данная однородная задача с однородными граничными условиями имеет только нулевое решение) относится к теории краевых задач для обыкновенных дифференциальных уравнений, которая не входит в программу курса численных методов. В настоящей работе данная проверка не приводится. В тестовых примерах $k(x) = x^2 + 1 \geq 1 > 0$, $q(x) > 0$ на $[0, 1]$, а коэффициенты граничных условий положительны. Сходимость численного решения для всех трёх задач с порядком $O(h^2)$ косвенно подтверждает корректность постановки.

3 Вывод разностных формул

В этом разделе подробно, шаг за шагом, показано, как из непрерывной краевой задачи получаются коэффициенты матрицы СЛАУ. Для построения консервативной разностной схемы используется **интегро-интерполяционный метод (метод баланса)** [3].

3.1 Интегро-интерполяционный метод (метод баланса)

Рассматривается уравнение

$$-(k(x)u'(x))' + q(x)u(x) = f(x), \quad a < x < b. \quad (3.1)$$

Введём равномерную сетку:

$$x_i = a + (i - 1)h, \quad i = 1, 2, \dots, N, \quad h = \frac{b - a}{N - 1}.$$

Обозначим: $u_i = u(x_i)$, $q_i = q(x_i)$, $f_i = f(x_i)$.

Введём поток $W(x) = -k(x)u'(x)$. Тогда уравнение переписывается как

$$-W' - qu = -f \quad \Rightarrow \quad W' + qu = f. \quad (3.2)$$

Проинтегрируем последнее на отрезке $[x_{i-1/2}, x_{i+1/2}]$, где $x_{i\pm 1/2} = x_i \pm h/2$:

$$\int_{x_{i-1/2}}^{x_{i+1/2}} W' dx + \int_{x_{i-1/2}}^{x_{i+1/2}} qu dx = \int_{x_{i-1/2}}^{x_{i+1/2}} f dx. \quad (3.3)$$

Первый интеграл вычисляется точно:

$$W(x_{i+1/2}) - W(x_{i-1/2}) + \int_{x_{i-1/2}}^{x_{i+1/2}} (qu - f) dx = 0. \quad (3.4)$$

Это уравнение баланса на элементарной ячейке.

3.2 Аппроксимация интегралов и потоков

Заменим интегралы по формуле прямоугольников, полагая подынтегральные функции постоянными на интервале интегрирования:

$$\int_{x_{i-1/2}}^{x_{i+1/2}} qu \, dx \approx q_i u_i h, \quad \int_{x_{i-1/2}}^{x_{i+1/2}} f \, dx \approx f_i h. \quad (3.5)$$

Для аппроксимации потоков $W_{i-1/2} = W(x_{i-1/2})$ и $W_{i+1/2} = W(x_{i+1/2})$ используем определение $W = -ku'$. Проинтегрируем равенство $u' = -W/k$ на отрезке $[x_{i-1}, x_i]$:

$$u_{i-1} - u_i = \int_{x_{i-1}}^{x_i} \frac{W}{k} dx. \quad (3.6)$$

Считая поток $W = W_{i-1/2}$ постоянным на этом интервале, получаем:

$$u_{i-1} - u_i \approx W_{i-1/2} \int_{x_{i-1}}^{x_i} \frac{dx}{k(x)}. \quad (3.7)$$

Отсюда выражаем поток:

$$W_{i-1/2} \approx -a_i \frac{u_i - u_{i-1}}{h}, \quad a_i = \left(\frac{1}{h} \int_{x_{i-1}}^{x_i} \frac{dx}{k(x)} \right)^{-1}. \quad (3.8)$$

Аналогично для правой границы ячейки:

$$W_{i+1/2} \approx -a_{i+1} \frac{u_{i+1} - u_i}{h}, \quad a_{i+1} = \left(\frac{1}{h} \int_{x_i}^{x_{i+1}} \frac{dx}{k(x)} \right)^{-1}. \quad (3.9)$$

Подставляя последние формулы в уравнение баланса, получаем:

$$-a_{i+1} \frac{u_{i+1} - u_i}{h} + a_i \frac{u_i - u_{i-1}}{h} + (q_i u_i - f_i) h = 0. \quad (3.10)$$

Разделим на h :

$$\frac{1}{h} \left(-a_{i+1} \frac{u_{i+1} - u_i}{h} + a_i \frac{u_i - u_{i-1}}{h} \right) + q_i u_i = f_i, \quad i = 2, \dots, N-1. \quad (3.11)$$

Если коэффициенты $k(x)$ являются гладкими функциями, то с точно-

стью до $O(h^2)$ можно положить:

$$a_i = k(x_i - h/2) \equiv k_{i-1/2}, \quad a_{i+1} = k(x_i + h/2) \equiv k_{i+1/2}.$$

Подставим эти выражения в уравнение:

$$\frac{1}{h} \left(-k_{i+1/2} \frac{u_{i+1} - u_i}{h} + k_{i-1/2} \frac{u_i - u_{i-1}}{h} \right) + q_i u_i = f_i.$$

Раскроем скобки:

$$\frac{-k_{i+1/2}(u_{i+1} - u_i) + k_{i-1/2}(u_i - u_{i-1})}{h^2} + q_i u_i = f_i.$$

Раскроем скобки в числителе:

$$\frac{-k_{i+1/2}u_{i+1} + k_{i+1/2}u_i + k_{i-1/2}u_i - k_{i-1/2}u_{i-1}}{h^2} + q_i u_i = f_i.$$

Сгруппируем слагаемые при u_{i-1} , u_i и u_{i+1} :

$$-\frac{k_{i-1/2}}{h^2}u_{i-1} + \left(\frac{k_{i-1/2} + k_{i+1/2}}{h^2} + q_i \right) u_i - \frac{k_{i+1/2}}{h^2}u_{i+1} = f_i. \quad (3.12)$$

Это основное разностное уравнение для внутренних узлов.

В программе оно реализовано следующим циклом:

```
for i = 2:N-1
    a_diag(i-1) = -kmid(i-1)/h^2;
    c_diag(i)   = -kmid(i)/h^2;
    b_diag(i)   = (kmid(i-1)+kmid(i))/h^2 + qv(i);
    d_vec(i)    = fv(i);
end
```

3.3 Аппроксимация граничных условий методом фиктивных узлов

Для сохранения второго порядка аппроксимации при учёте граничных условий второго и третьего рода используется метод фиктивных узлов (законтурных точек), подробно описанный в [4]. Сеточная область продолжается за пределы отрезка $[a, b]$: вводятся дополнительные точки

$$x_0 = a - h, \quad x_{N+1} = b + h,$$

со значениями искомой функции $u_0 = u(x_0)$ и $u_{N+1} = u(x_{N+1})$. Производные на границах аппроксимируются центральными разностями, что обеспечивает второй порядок точности:

$$u'(a) \approx \frac{u_2 - u_0}{2h}, \quad u'(b) \approx \frac{u_{N+1} - u_{N-1}}{2h}. \quad (3.13)$$

Далее фиктивные значения u_0 и u_{N+1} исключаются путём выражения их из соответствующих граничных условий и подстановки в разностные уравнения для крайних узлов $i = 1$ и $i = N$.

3.3.1 Левая граница: условие Неймана

Пусть задано $u'(a) = \gamma$. Из формулы (3.13) для производной выразим u_0 :

$$\frac{u_2 - u_0}{2h} = \gamma \quad \Rightarrow \quad u_0 = u_2 - 2h\gamma.$$

Запишем уравнение (3.12) для левого граничного узла $i = 1$. При этом $k_{i-1/2} = k(a - h/2) \equiv k_{-1/2}$, $k_{i+1/2} = k(a + h/2) \equiv k_{1/2}$:

$$-\frac{k_{-1/2}}{h^2}u_0 + \left(\frac{k_{-1/2} + k_{1/2}}{h^2} + q_1 \right) u_1 - \frac{k_{1/2}}{h^2}u_2 = f_1.$$

Подставляем выражение для u_0 :

$$-\frac{k_{-1/2}}{h^2}(u_2 - 2h\gamma) + \left(\frac{k_{-1/2} + k_{1/2}}{h^2} + q_1 \right) u_1 - \frac{k_{1/2}}{h^2}u_2 = f_1.$$

Раскрываем скобки и группируем слагаемые:

$$-\frac{k_{-1/2} + k_{1/2}}{h^2}u_2 + \left(\frac{k_{-1/2} + k_{1/2}}{h^2} + q_1 \right) u_1 + \frac{2k_{-1/2}}{h}\gamma = f_1.$$

Переносим известное слагаемое с γ в правую часть:

$$\left(\frac{k_{-1/2} + k_{1/2}}{h^2} + q_1 \right) u_1 - \frac{k_{-1/2} + k_{1/2}}{h^2}u_2 = f_1 - \frac{2k_{-1/2}}{h}\gamma. \quad (3.14)$$

В программе это соответствует:

```
k_minus_half = k(a - h/2);
k_plus_half = k(a + h/2);
b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1);
```

```

c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
d_vec(1)  = fv(1) - 2*k_minus_half/h * gamma;

```

3.3.2 Левая граница: условие Робина

Пусть задано $\alpha_0 u(a) + \alpha_1 u'(a) = \gamma_a$. Подставляем аппроксимацию производной:

$$\alpha_0 u_1 + \alpha_1 \cdot \frac{u_2 - u_0}{2h} = \gamma_a.$$

Выражаем u_0 :

$$\alpha_1 \cdot \frac{u_2 - u_0}{2h} = \gamma_a - \alpha_0 u_1 \quad \Rightarrow \quad u_2 - u_0 = \frac{2h}{\alpha_1} (\gamma_a - \alpha_0 u_1) \quad \Rightarrow \quad u_0 = u_2 - \frac{2h}{\alpha_1} (\gamma_a - \alpha_0 u_1).$$

Подставляем в уравнение для $i = 1$:

$$-\frac{k_{-1/2}}{h^2} \left[u_2 - \frac{2h}{\alpha_1} (\gamma_a - \alpha_0 u_1) \right] + \left(\frac{k_{-1/2} + k_{1/2}}{h^2} + q_1 \right) u_1 - \frac{k_{1/2}}{h^2} u_2 = f_1.$$

Раскрываем скобки и приводим подобные:

$$-\frac{k_{-1/2}}{h^2} u_2 + \frac{2k_{-1/2}}{h\alpha_1} \gamma_a - \frac{2k_{-1/2}\alpha_0}{h\alpha_1} u_1 + \left(\frac{k_{-1/2} + k_{1/2}}{h^2} + q_1 \right) u_1 - \frac{k_{1/2}}{h^2} u_2 = f_1.$$

Объединяем слагаемые при u_1 и u_2 :

$$-\frac{k_{-1/2} + k_{1/2}}{h^2} u_2 + \left(\frac{k_{-1/2} + k_{1/2}}{h^2} + q_1 - \frac{2k_{-1/2}\alpha_0}{h\alpha_1} \right) u_1 + \frac{2k_{-1/2}}{h\alpha_1} \gamma_a = f_1.$$

Переносим известное слагаемое в правую часть:

$$\left(\frac{k_{-1/2} + k_{1/2}}{h^2} + q_1 - \frac{2k_{-1/2}\alpha_0}{h\alpha_1} \right) u_1 - \frac{k_{-1/2} + k_{1/2}}{h^2} u_2 = f_1 - \frac{2k_{-1/2}}{h\alpha_1} \gamma_a. \quad (3.15)$$

В программе это соответствует:

```

k_minus_half = k(a - h/2);
k_plus_half  = k(a + h/2);
b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1) - 2*k_minus_half*alpha0/(h*alpha1);
c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
d_vec(1)  = fv(1) - 2*k_minus_half/(h*alpha1)*gamma_a;

```

3.3.3 Правая граница: условие Неймана

Для правой границы $i = N$ с условием $u'(b) = \delta$ выражаем u_{N+1} :

$$\frac{u_{N+1} - u_{N-1}}{2h} = \delta \quad \Rightarrow \quad u_{N+1} = u_{N-1} + 2h\delta.$$

Записываем уравнение (3.12) для $i = N$, где $k_{N-1/2} = k(b - h/2)$, $k_{N+1/2} = k(b + h/2)$:

$$-\frac{k_{N-1/2}}{h^2}u_{N-1} + \left(\frac{k_{N-1/2} + k_{N+1/2}}{h^2} + q_N\right)u_N - \frac{k_{N+1/2}}{h^2}u_{N+1} = f_N.$$

Подставляем выражение для u_{N+1} :

$$-\frac{k_{N-1/2}}{h^2}u_{N-1} + \left(\frac{k_{N-1/2} + k_{N+1/2}}{h^2} + q_N\right)u_N - \frac{k_{N+1/2}}{h^2}(u_{N-1} + 2h\delta) = f_N.$$

Раскрываем скобки и группируем:

$$-\frac{k_{N-1/2} + k_{N+1/2}}{h^2}u_{N-1} + \left(\frac{k_{N-1/2} + k_{N+1/2}}{h^2} + q_N\right)u_N - \frac{2k_{N+1/2}}{h}\delta = f_N.$$

Переносим известное слагаемое в правую часть:

$$-\frac{k_{N-1/2} + k_{N+1/2}}{h^2}u_{N-1} + \left(\frac{k_{N-1/2} + k_{N+1/2}}{h^2} + q_N\right)u_N = f_N + \frac{2k_{N+1/2}}{h}\delta. \quad (3.16)$$

В программе это соответствует:

```
k_minus_half = k(b - h/2);
k_plus_half = k(b + h/2);
a_diag(N-1) = -(k_minus_half + k_plus_half)/h^2;
b_diag(N) = (k_minus_half + k_plus_half)/h^2 + qv(N);
d_vec(N) = fv(N) + 2*k_plus_half/h * delta;
```

3.3.4 Правая граница: условие Робина

Для условия $\beta_0 u(b) + \beta_1 u'(b) = \gamma_b$ выражаем u_{N+1} :

$$\beta_0 u_N + \beta_1 \cdot \frac{u_{N+1} - u_{N-1}}{2h} = \gamma_b \quad \Rightarrow \quad u_{N+1} = u_{N-1} + \frac{2h}{\beta_1}(\gamma_b - \beta_0 u_N).$$

Подставляем в уравнение для $i = N$ и выполняем преобразования,

аналогичные случаю левой границы:

$$-\frac{k_{N-1/2} + k_{N+1/2}}{h^2}u_{N-1} + \left(\frac{k_{N-1/2} + k_{N+1/2}}{h^2} + q_N + \frac{2k_{N+1/2}\beta_0}{h\beta_1} \right) u_N = f_N + \frac{2k_{N+1/2}}{h\beta_1}\gamma_b. \quad (3.17)$$

В программе это соответствует:

```
k_minus_half = k(b - h/2);
k_plus_half = k(b + h/2);
a_diag(N-1) = -(k_minus_half + k_plus_half)/h^2;
b_diag(N) = (k_minus_half + k_plus_half)/h^2 + qv(N) + 2*k_plus_half*beta0/(h*beta1);
d_vec(N) = fv(N) + 2*k_plus_half/(h*beta1)*gamma_b;
```

3.4 Условия Дирихле

Для условий первого рода граничные значения известны: $u_1 = A$, $u_N = B$. Поэтому первое и последнее уравнения системы заменяются на тождества:

$$u_1 = A, \quad u_N = B.$$

В программе это реализовано так:

```
b_diag(1) = 1; c_diag(1) = 0; d_vec(1) = A;
a_diag(N-1) = 0; b_diag(N) = 1; d_vec(N) = B;
```

3.5 Итоговая система

После описанных преобразований получается система линейных алгебраических уравнений с трёхдиагональной матрицей, которая эффективно решается методом прогонки.

4 Метод прогонки решения трёхдиагональной СЛАУ

Алгоритм решения системы с трёхдиагональной матрицей, определяемый формулами прямого и обратного хода, называется методом прогонки и подробно изложен в [5].

В результате дискретизации получена система линейных алгебраических уравнений с трёхдиагональной матрицей:

$$\begin{cases} b_1 u_1 + c_1 u_2 = d_1, \\ a_2 u_1 + b_2 u_2 + c_2 u_3 = d_2, \\ \dots \\ a_i u_{i-1} + b_i u_i + c_i u_{i+1} = d_i, \quad i = 2, 3, \dots, N-1, \\ \dots \\ a_N u_{N-1} + b_N u_N = d_N, \end{cases}$$

где a_i, b_i, c_i — коэффициенты, d_i — правая часть.

Для решения такой системы используется **метод прогонки** — частный случай метода Гаусса, учитывающий трёхдиагональную структуру матрицы и имеющий вычислительную сложность $O(N)$.

4.1 Алгоритм метода прогонки

Прямой ход прогонки (вычисление прогоночных коэффициентов α_i и β_i):

$$\alpha_1 = \frac{c_1}{b_1}, \quad \beta_1 = \frac{d_1}{b_1},$$
$$\alpha_i = \frac{c_i}{b_i - a_i \alpha_{i-1}}, \quad \beta_i = \frac{d_i - a_i \beta_{i-1}}{b_i - a_i \alpha_{i-1}}, \quad i = 2, 3, \dots, N-1.$$

Обратный ход (восстановление решения):

$$u_N = \frac{d_N - a_N \beta_{N-1}}{b_N - a_N \alpha_{N-1}},$$
$$u_i = \beta_i - \alpha_i u_{i+1}, \quad i = N-1, N-2, \dots, 1.$$

Условия корректности метода (достаточное условие устойчивости):

$$|b_i| \geq |a_i| + |c_i|, \quad i = 1, \dots, N,$$

причём хотя бы для одного i неравенство строгое. Для рассматриваемой краевой задачи эти условия выполняются, так как $k_{i\pm 1/2} > 0$ и $q_i \geq 0$.

В программной реализации используется упрощённый вариант прогонки.

5 Реализация метода конечных разностей для задачи с граничными условиями первого рода (Дирихле)

Рассматривается краевая задача:

$$-\frac{d}{dx} \left((x^2 + 1) \frac{du}{dx} \right) + (x + 1)u = x^5 - 21x^4 + 23x^3 - 14x^2 + 9x - 2, \quad 0 < x < 1,$$

с граничными условиями Дирихле:

$$u(0) = u_{\text{exact}}(0) = 0, \quad u(1) = u_{\text{exact}}(1) = 3.$$

Точное решение задачи:

$$u_{\text{exact}}(x) = x^4 - 2x^3 + x^2 + 3x.$$

Параметры задачи: $k(x) = x^2 + 1$, $q(x) = x + 1$, $f(x) = x^5 - 21x^4 + 23x^3 - 14x^2 + 9x - 2$.

Программная реализация метода для задачи Дирихле приведена в Приложении 1. Ниже представлены результаты вычислительного эксперимента.

5.1 Результаты сгущения сетки

В процессе решения сетка автоматически сгущалась. Результаты приведены в таблице 1.

Таблица 1: Сходимость метода при сгущении сетки

N	h	Оценка погрешности (правило Рунге)
9	0.125000	3.374646×10^{-3}
17	0.062500	8.679494×10^{-4}
33	0.031250	2.195086×10^{-4}
65	0.015625	5.504119×10^{-5}

Оценка погрешности проводилась по правилу Рунге:

$$\text{err}_{\text{est}} = \frac{\max |U_{\text{coarse}} - U_{\text{prev}}|}{2^p - 1} = \frac{\max |U_{\text{coarse}} - U_{\text{prev}}|}{3}, \quad p = 2.$$

5.2 График численного и точного решений

На рисунке 1 представлено численное и точное решения.

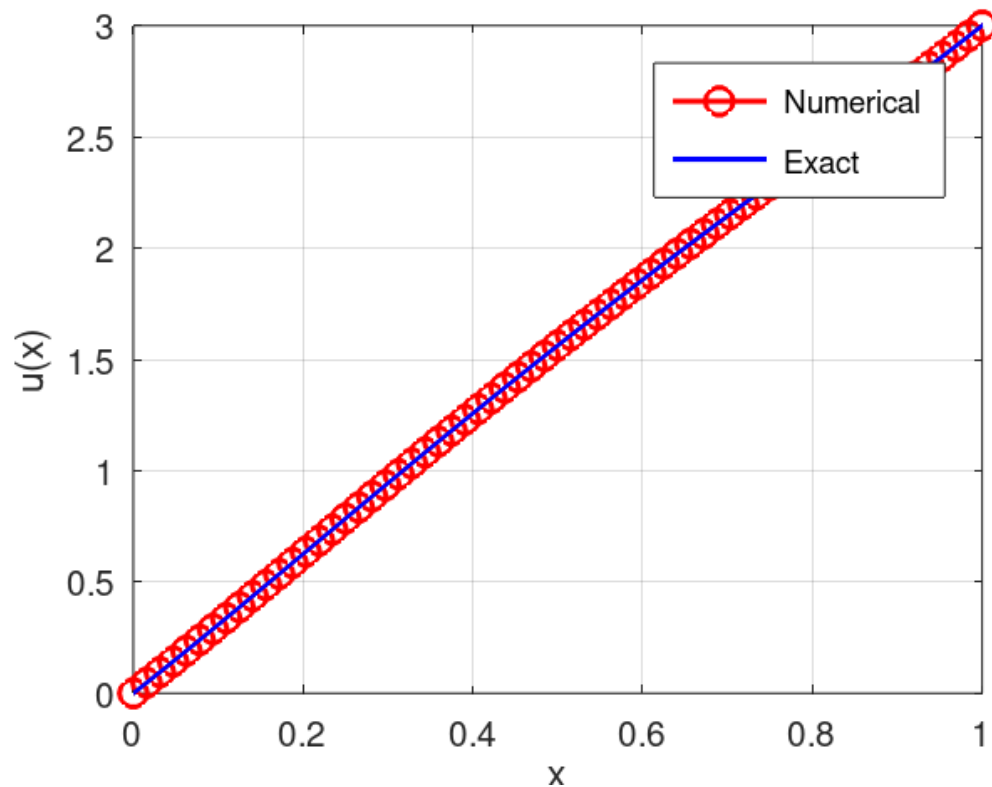


Рис. 1: Численное решение задачи Дирихле

5.3 График оценки погрешности

График распределения оценки погрешности по правилу Рунге показан на рисунке 2.

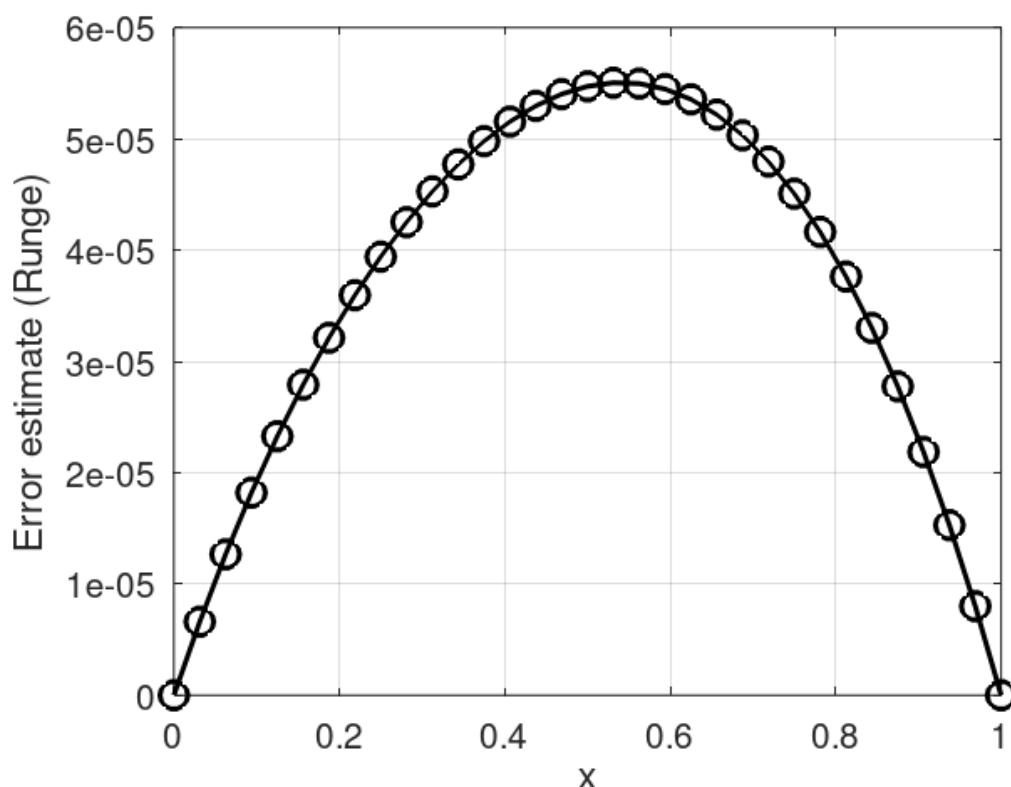


Рис. 2: Распределение оценки погрешности по узлам сетки

Эксперимент 1. Разрывной коэффициент $k(x)$

Была решена задача Дирихле, но с другим коэффициентом

$$k(x) = \begin{cases} 1, & 0 \leq x \leq 0.5, \\ 101, & 0.5 < x \leq 1. \end{cases}$$

то есть со скачком в 100 раз в середине отрезка (остальные параметры не менялись). Результаты представлены на рисунках 3 и 4.

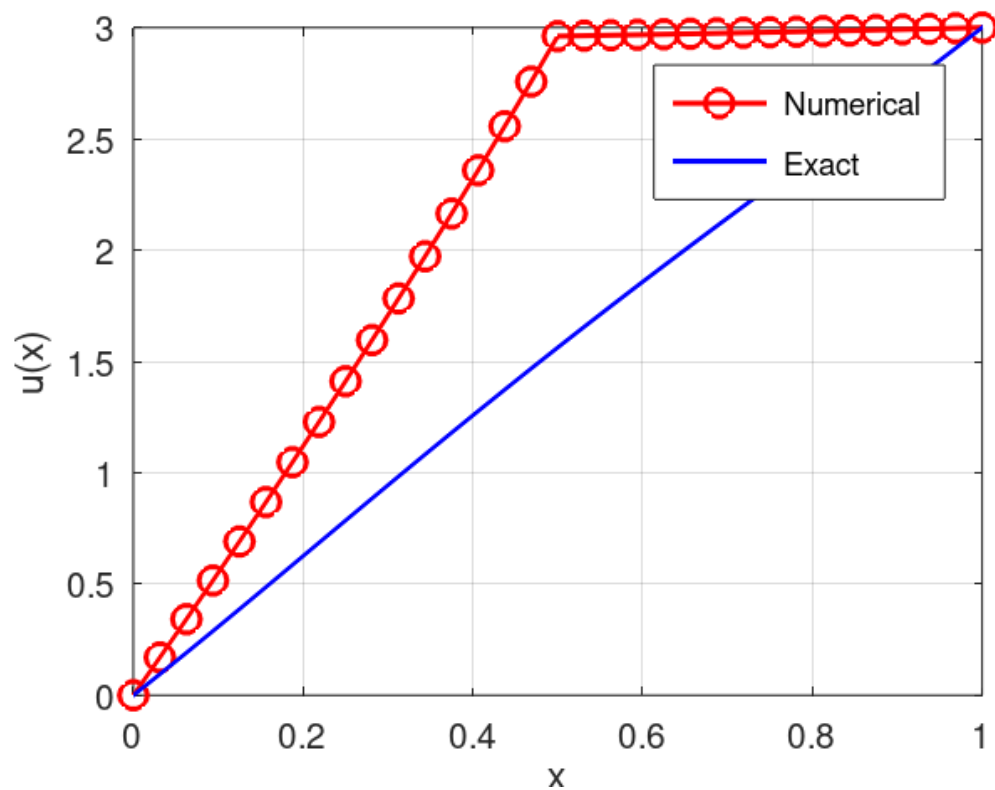


Рис. 3: Численное решение с разрывным коэффициентом

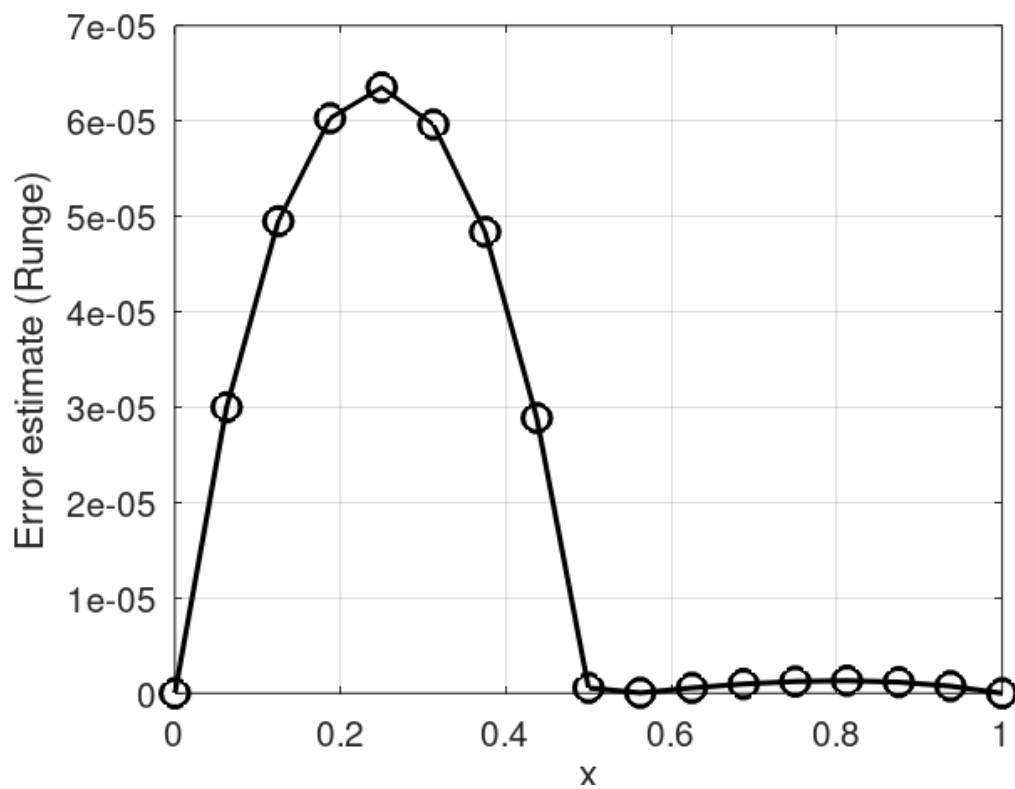


Рис. 4: Оценка погрешности для задачи с разрывным коэффициентом

Эксперимент 2. Влияние осциллирующей правой части

Для исследования влияния осцилляций (колебаний) правой части на численное решение была рассмотрена задача Дирихле с правой частью

$$f(x) = \sin(50\pi x),$$

нулевыми граничными условиями $u(0) = 0$, $u(1) = 0$ и уменьшенным коэффициентом

$$k(x) = 10^{-4}(x^2 + 1).$$

Остальные параметры задачи ($q(x) = x + 1$) оставлены без изменений.

На рисунках 5 и 6 представлены результаты численного решения.

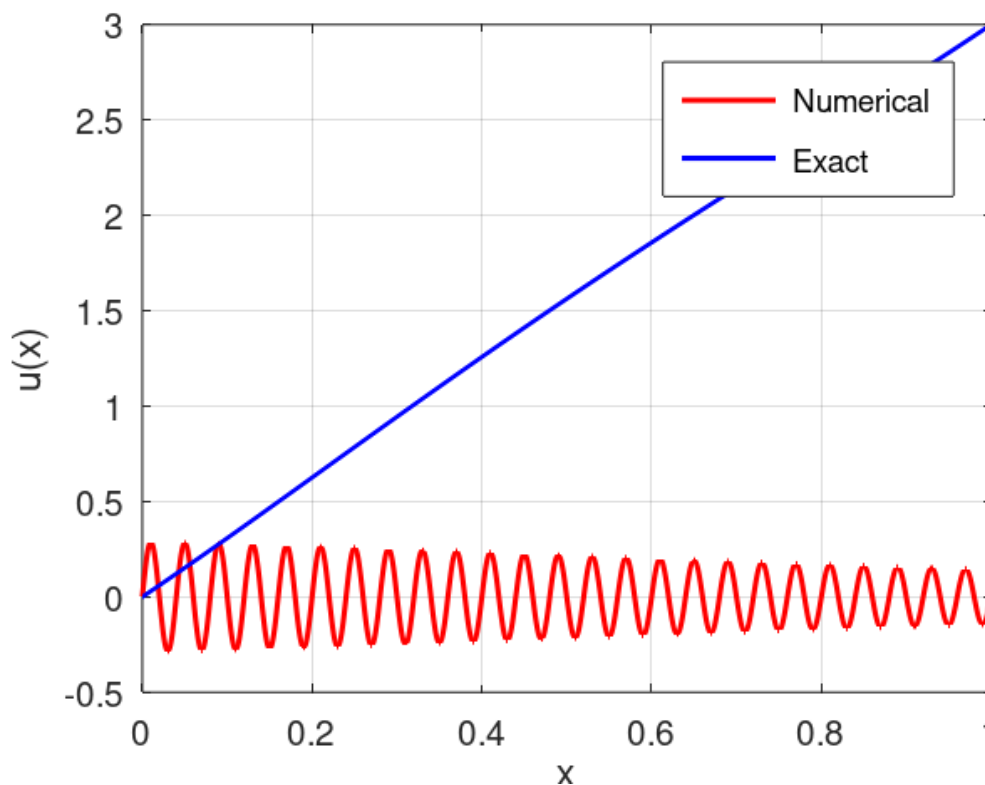


Рис. 5: Численное и точное решения задачи с осциллирующей правой частью

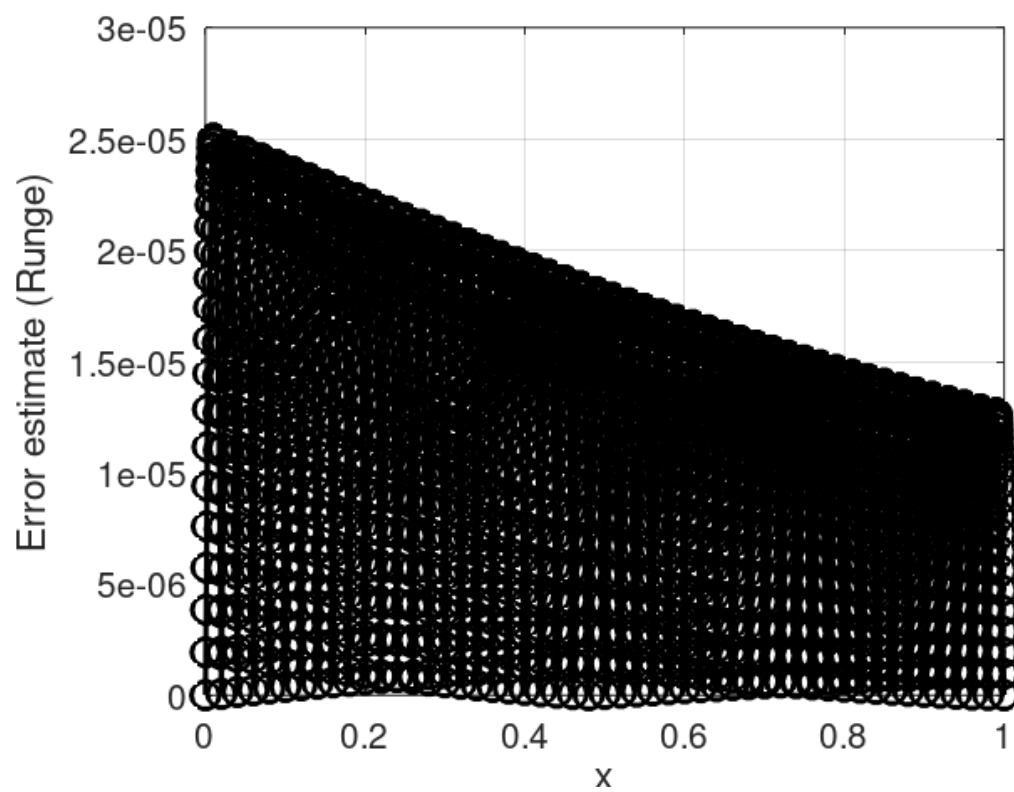


Рис. 6: Оценка погрешности по правилу Рунге

6 Реализация метода конечных разностей для задачи с граничными условиями второго рода (Неймана)

Рассматривается краевая задача:

$$-\frac{d}{dx} \left((x^2 + 1) \frac{du}{dx} \right) + 10(x+1)u = 10x^5 - 30x^4 + 14x^3 + 22x^2 + 36x - 2, \quad 0 < x < 1,$$

с граничными условиями Неймана:

$$u'(0) = \gamma = 3, \quad u'(1) = \delta = 3.$$

Точное решение задачи:

$$u_{\text{exact}}(x) = x^4 - 2x^3 + x^2 + 3x.$$

Параметры задачи: $k(x) = x^2 + 1$, $q(x) = 10(x + 1)$, $f(x) = 10x^5 - 30x^4 + 14x^3 + 22x^2 + 36x - 2$, $\gamma = 3$, $\delta = 3$.

Программная реализация метода для задачи Неймана приведена в Приложении 2. Ниже представлены результаты вычислительного эксперимента.

6.1 Результаты сгущения сетки

Процесс сгущения сетки продолжался до достижения оценки погрешности менее $\varepsilon = 10^{-4}$.

Таблица 2: Сходимость метода при сгущении сетки (задача Неймана)

N	h	Оценка погрешности (правило Рунге)
9	0.125000	6.500893×10^{-3}
17	0.062500	1.762322×10^{-3}
33	0.031250	4.499586×10^{-4}
65	0.015625	1.130875×10^{-4}
129	0.007812	2.830943×10^{-5}

6.2 График численного и точного решений

На рисунке 7 представлено численное и точное решения.

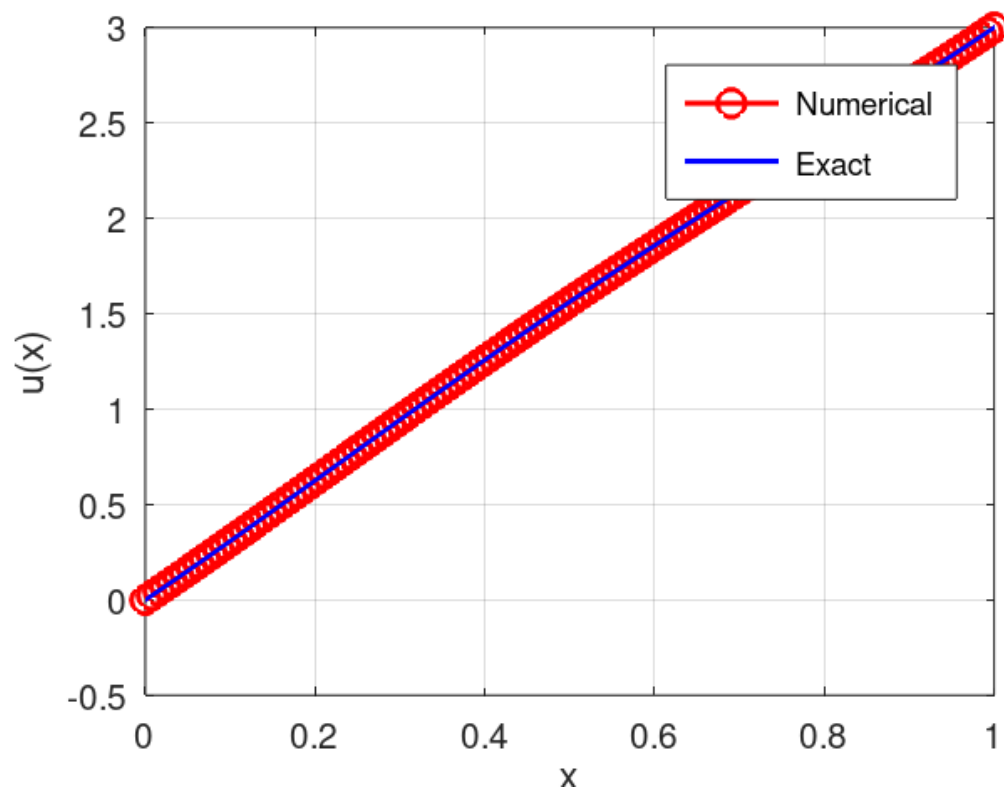


Рис. 7: Численное решение задачи Неймана

6.3 График оценки погрешности

График распределения оценки погрешности по правилу Рунге показан на рисунке 8.

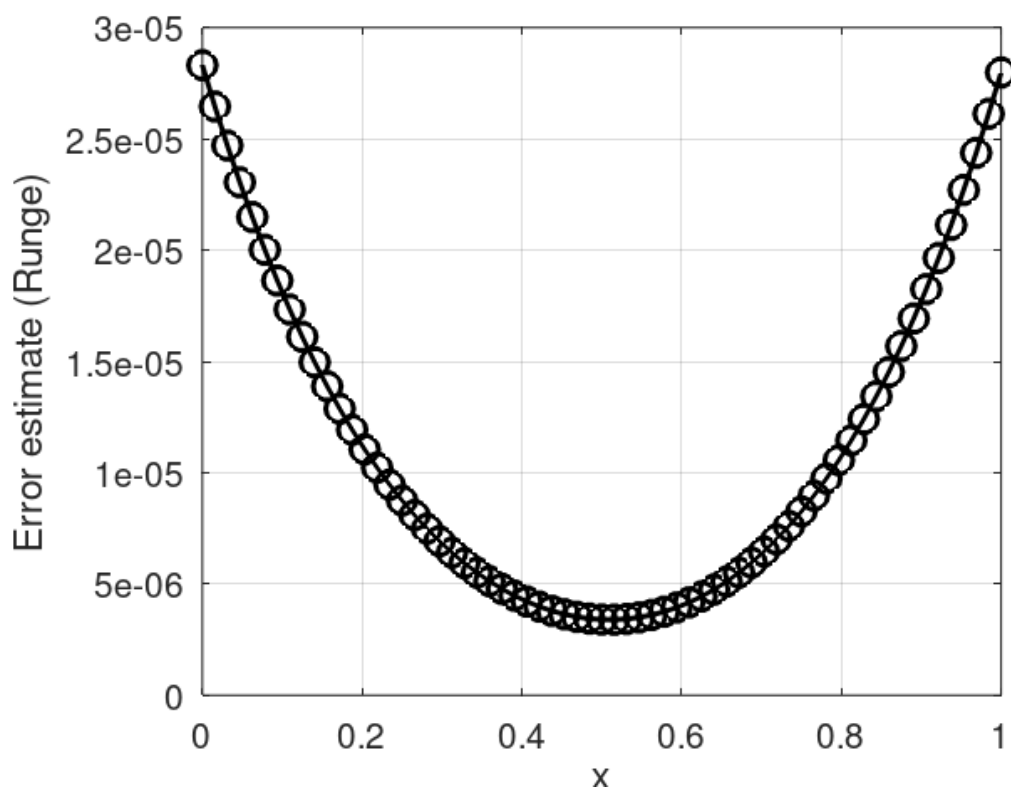


Рис. 8: Распределение оценки погрешности по узлам сетки

Эксперимент 3. Сравнение граничных условий Неймана

Исследовалось влияние граничных условий Неймана на решение при различных значениях производных на границах: $u'(0) = \gamma$ и $u'(1) = \delta$. Рассмотрены четыре комбинации: $(\gamma, \delta) = (0, 0)$, $(2, 0)$, $(0, 2)$, $(2, 2)$.

На рисунках красными цветом показано численное решение, синим — точное решение.

Код программы для данного эксперимента приведён в Приложении 4.

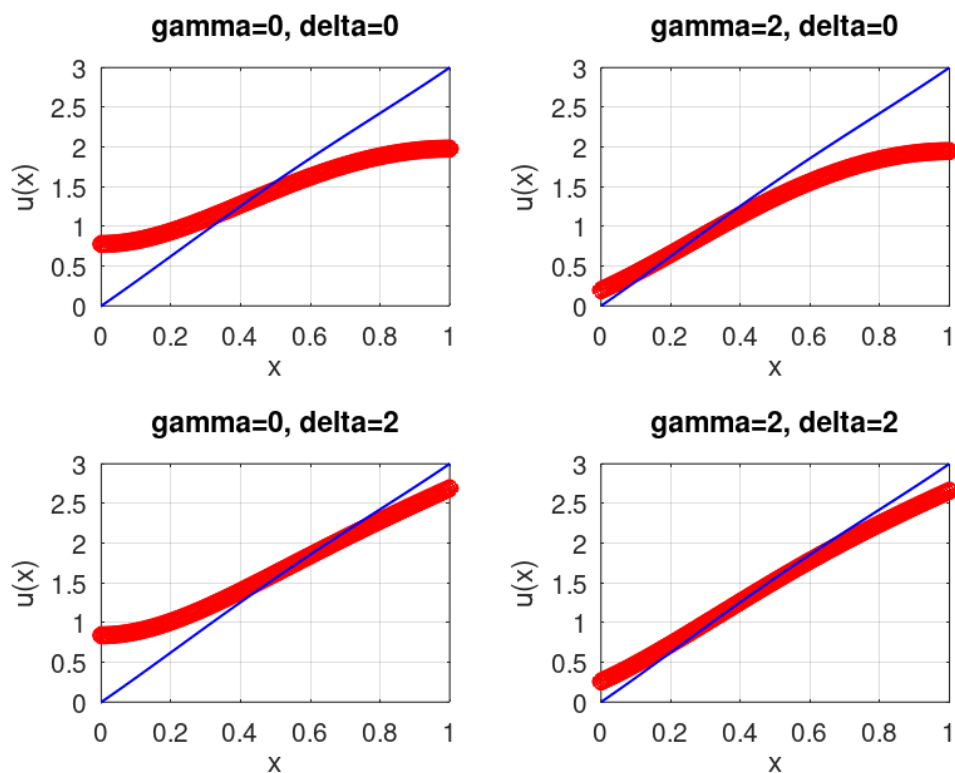


Рис. 9: Сравнение численных решений при различных граничных условиях Неймана

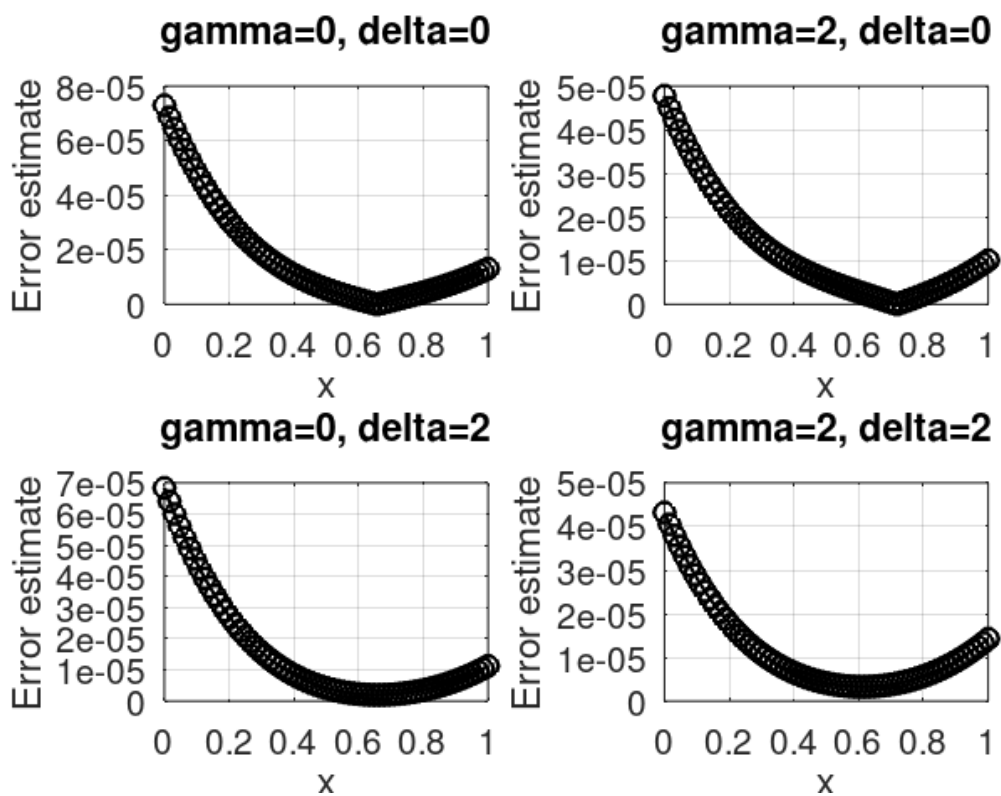


Рис. 10: Сравнение оценок погрешности при различных граничных условиях Неймана

7 Реализация метода конечных разностей для задачи с граничными условиями третьего рода (Робина)

Рассматривается краевая задача:

$$-\frac{d}{dx} \left((x^2 + 1) \frac{du}{dx} \right) + (x + 1)u = x^5 - 21x^4 + 23x^3 - 14x^2 + 9x - 2, \quad 0 < x < 1,$$

с граничными условиями Робина:

$$\alpha_0 u(0) + \alpha_1 u'(0) = \gamma_a, \quad \beta_0 u(1) + \beta_1 u'(1) = \gamma_b,$$

где $\alpha_0 = 1$, $\alpha_1 = 1$, $\gamma_a = 3$, $\beta_0 = 2$, $\beta_1 = 1$, $\gamma_b = 9$. Точное решение задачи:

$$u_{\text{exact}}(x) = x^4 - 2x^3 + x^2 + 3x.$$

Параметры задачи: $k(x) = x^2 + 1$, $q(x) = x + 1$, $f(x) = x^5 - 21x^4 + 23x^3 - 14x^2 + 9x - 2$, $\alpha_0 = 1$, $\alpha_1 = 1$, $\gamma_a = 3$, $\beta_0 = 2$, $\beta_1 = 1$, $\gamma_b = 9$.

Программная реализация метода для задачи Робина приведена в Приложении 3. Ниже представлены результаты вычислительного эксперимента.

7.1 Результаты сгущения сетки

Процесс сгущения сетки продолжался до достижения оценки погрешности менее $\varepsilon = 10^{-4}$.

Таблица 3: Сходимость метода при сгущении сетки (задача Робина)

N	h	Оценка погрешности (правило Рунге)
9	0.125000	5.700689×10^{-2}
17	0.062500	1.392823×10^{-2}
33	0.031250	3.462401×10^{-3}
65	0.015625	8.643804×10^{-4}
129	0.007812	2.160190×10^{-4}
257	0.003906	5.399999×10^{-5}

7.2 График численного и точного решений

На рисунке 11 представлено численное и точное решения.

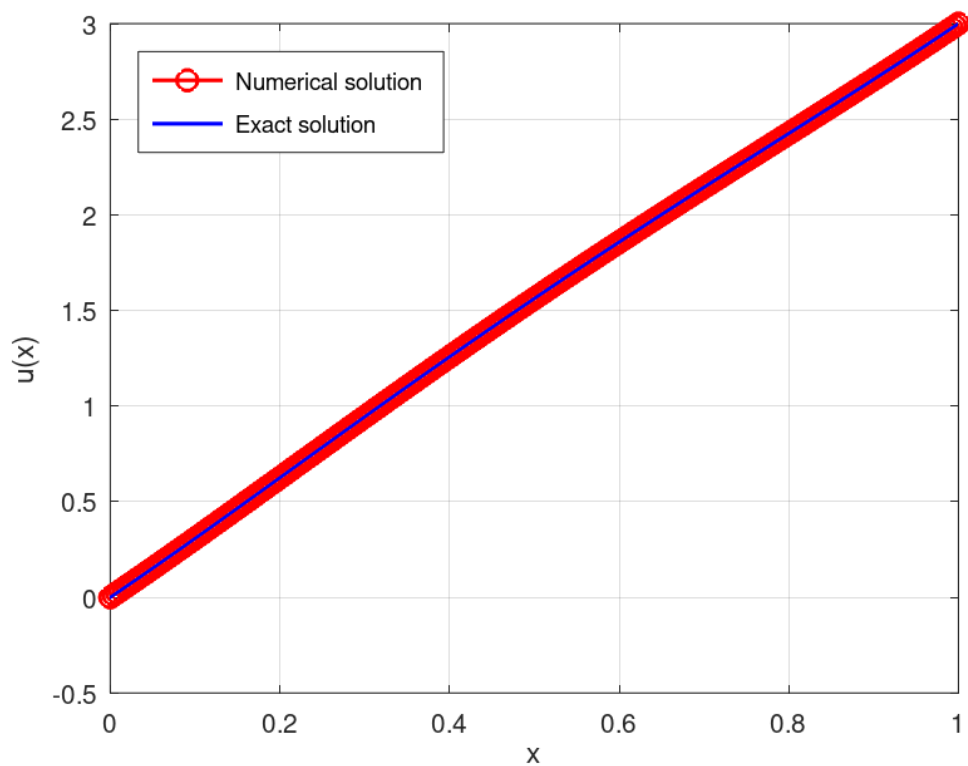


Рис. 11: Численное решение задачи с условиями Робина

7.3 График оценки погрешности

График распределения оценки погрешности по правилу Рунге показан на рисунке 12.

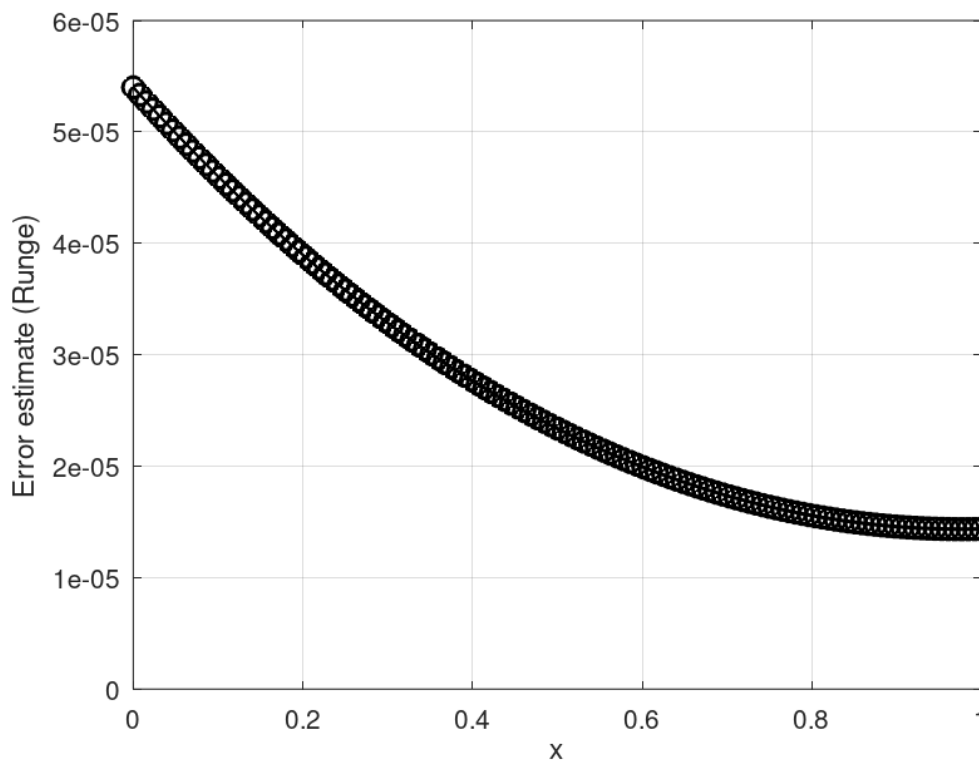


Рис. 12: Распределение оценки погрешности по узлам сетки

Эксперимент 4. Зависимость количества итераций от требуемой точности

На рисунке 13 представлена зависимость числа итераций от требуемой точности.

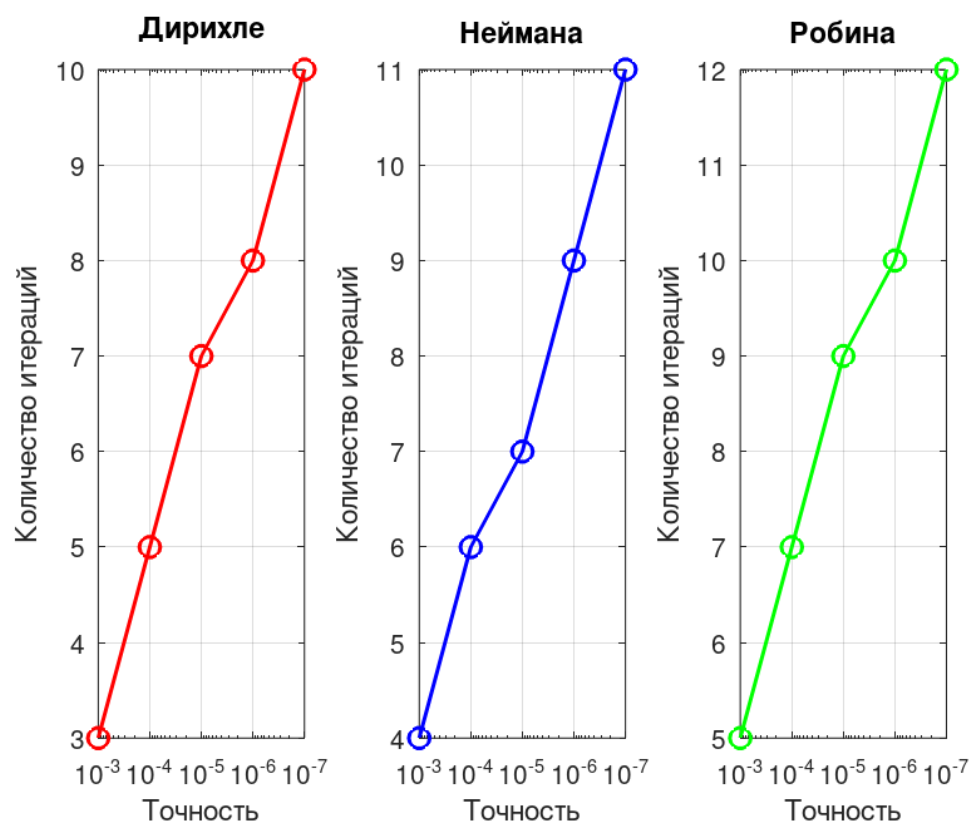


Рис. 13: Зависимость количества итераций от требуемой точности

Заключение

В ходе выполнения курсовой работы реализован метод конечных разностей для решения линейной двухточечной краевой задачи для ОДУ второго порядка с переменными коэффициентами. Была описана математическая постановка задачи, проведён вывод разностных формул на основе интегро-интерполяционного метода (метода баланса), построена консервативная разностная схема второго порядка аппроксимации. Для решения полученной трёхдиагональной СЛАУ применён метод прогонки.

Для всех трёх типов граничных условий (Дирихле, Неймана, Робина) проведены вычислительные эксперименты с автоматическим сгущением сетки до достижения точности 10^{-4} . Оценка погрешности по правилу Рунге подтвердила второй порядок сходимости разностной схемы. Построены графики численного решения и распределения оценки погрешности по узлам сетки.

Требуемая точность достигнута при $N = 65$ (Дирихле), $N = 129$ (Неймана) и $N = 257$ (Робина).

Список литературы

- [1] Амосов А. А., Дубинский Ю. А., Копченова Н. В. Вычислительные методы для инженеров : учеб. пособие. — М. : Высшая школа, 1994. — 544 с.
- [2] Денисов А. М., Разгулин А. В. Обыкновенные дифференциальные уравнения. Часть 2 : учеб. пособие. — М. : Факультет вычислительной математики и кибернетики МГУ им. М. В. Ломоносова, 2009. — 114 с.
- [3] Методы построения разностных схем : учеб.-метод. материал / М. : Физический факультет МГУ им. М. В. Ломоносова, 2014. — Режим доступа: https://math.phys.msu.ru/archive/2014_2015/197/metodi%20postroeniya.pdf.
- [4] Марчук Г. И. Методы вычислительной математики. — 2-е изд., перераб. и доп. — М. : Наука, 1980. — 536 с.
- [5] Самарский А. А., Гулин А. В. Численные методы : учеб. пособие для вузов. — М. : Наука, 1989. — 432 с.

Приложение

Приложение 1. Код программы для задачи Дирихле

```
graphics_toolkit("qt");

% Определяем границы интервала
a = 0; b = 1;

% Коэффициенты дифференциального уравнения
k = @(x) x.^2 + 1;
q = @(x) x + 1;
f = @(x) x.^5 - 21*x.^4 + 23*x.^3 - 14*x.^2 + 9*x - 2;

% Точное решение и граничные условия
u_exact = @(x) x.^4 - 2*x.^3 + x.^2 + 3*x;
A = u_exact(a);
B = u_exact(b);

% Параметры метода
tol = 1e-4; % Требуемая точность
N = 5; % Начальное число узлов
err_est = tol + 1;
first = true;

% Основной цикл автоматического сгущения сетки
while err_est > tol
    h = (b - a)/(N - 1); % Шаг сетки
    x = linspace(a, b, N)'; % Узлы сетки
    xmid = (x(1:end-1) + x(2:end))/2; % Середины отрезков

    kmid = k(xmid);
    qv = q(x);
    fv = f(x);

    % Заполнение трехдиагональной матрицы
    a_diag = zeros(N,1); % Нижняя диагональ
    b_diag = zeros(N,1); % Главная диагональ
    c_diag = zeros(N,1); % Верхняя диагональ
    d_vec = zeros(N,1); % Вектор правой части

    % Внутренние узлы
    for i = 2:N-1
        a_diag(i-1) = -kmid(i-1)/h^2;
        c_diag(i) = -kmid(i)/h^2;
        b_diag(i) = (kmid(i-1) + kmid(i))/h^2 + qv(i);
        d_vec(i) = fv(i);
    end

    % Граничные условия (Дирихле)
    b_diag(1) = 1; c_diag(1) = 0; d_vec(1) = A;
    a_diag(N-1) = 0; b_diag(N) = 1; d_vec(N) = B;

    % Прогонка (решение трехдиагональной системы)
    n = length(d_vec);
    aa = a_diag(1:n-1);
    bb = b_diag;
    cc = c_diag(1:n-1);
    dd = d_vec;
```

```

% Прямой ход прогонки
for i = 2:n
    w = aa(i-1) / bb(i-1);
    bb(i) = bb(i) - w * cc(i-1);
    dd(i) = dd(i) - w * dd(i-1);
end

% Обратный ход прогонки
U = zeros(n,1);
U(n) = dd(n) / bb(n);
for i = n-1:-1:1
    U(i) = (dd(i) - cc(i) * U(i+1)) / bb(i);
end

% Оценка ошибки по Рунге
if ~first
    U_fine_on_coarse_nodes = U(1:2:end); % Берем каждую вторую точку
    err_est = max(abs(U_fine_on_coarse_nodes - U_prev)) / 3;
    fprintf('N = %d, h = %.6f, err_est = %.6e\n', N, h, err_est);

    if err_est < tol % Достигнута нужная точность
        x_final = x;
        U_final = U;
        U_prev_for_plot = U_prev;
        x_prev_for_plot = x_prev;
        break;
    end
else
    first = false; % Первый шаг - просто сохраняем решение
end

% Удваиваем число узлов для следующей итерации
U_prev = U;
x_prev = x;
N = 2*N - 1;
end

fprintf('Runge error estimate: %.6e\n', err_est);

% График численного и точного решения
figure;
plot(x_final, U_final, 'ro-', 'MarkerSize', 8, 'LineWidth', 1.5);
hold on;
plot(x_final, u_exact(x_final), 'b-', 'LineWidth', 1.5);
hold off;
xlabel('x'); ylabel('u(x)');
legend('Numerical', 'Exact');
grid on;

% График оценки ошибки по Рунге
figure;
x_coarse = x_prev_for_plot;
U_coarse = U_prev_for_plot;
U_fine_at_coarse_nodes = U_final(1:2:end);

error_estimate = abs(U_fine_at_coarse_nodes - U_coarse) / 3;
plot(x_coarse, error_estimate, 'ko-', 'MarkerSize', 8, 'LineWidth', 1.5);
xlabel('x'); ylabel('Error estimate (Runge)');
grid on;

```

Приложение 2. Код программы для задачи Неймана

```
graphics_toolkit("qt");

% Определяем границы интервала
a = 0; b = 1;

% Коэффициенты дифференциального уравнения
k = @(x) x.^2 + 1;
q = @(x) 10*(x + 1);
f = @(x) 10*x.^5 - 30*x.^4 + 14*x.^3 + 22*x.^2 + 36*x - 2;

% Параметры граничных условий
gamma = 3;
delta = 3;

% Параметры метода
tol = 1e-4;          % Требуемая точность
N = 5;              % Начальное число узлов
err_est = tol + 1;
first = true;

% Основной цикл автоматического сгущения сетки
while err_est > tol
    h = (b - a)/(N - 1);          % Шаг сетки
    x = linspace(a, b, N)';      % Узлы сетки
    xmid = (x(1:end-1) + x(2:end))/2; % Середины отрезков

    kmid = k(xmid);
    qv = q(x);
    fv = f(x);

    % Заполнение трехдиагональной матрицы
    a_diag = zeros(N,1); % Нижняя диагональ
    b_diag = zeros(N,1); % Главная диагональ
    c_diag = zeros(N,1); % Верхняя диагональ
    d_vec = zeros(N,1);  % Вектор правой части

    % Внутренние узлы
    for i = 2:N-1
        a_diag(i-1) = -kmid(i-1)/h^2;
        c_diag(i)   = -kmid(i)/h^2;
        b_diag(i)   = (kmid(i-1) + kmid(i))/h^2 + qv(i);
        d_vec(i)    = fv(i);
    end

    % Левое граничное условие
    k_minus_half = k(a - h/2);
    k_plus_half = k(a + h/2);
    b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1);
    c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
    d_vec(1) = fv(1) - 2*k_minus_half/h * gamma;

    % Правое граничное условие
    k_minus_half = k(b - h/2);
    k_plus_half = k(b + h/2);
    a_diag(N-1) = -(k_minus_half + k_plus_half)/h^2;
    b_diag(N) = (k_minus_half + k_plus_half)/h^2 + qv(N);
    d_vec(N) = fv(N) + 2*k_plus_half/h * delta;
```

```

% Решение системы методом прогонки
aa = a_diag(1:N-1);
cc = c_diag(1:N-1);
n = length(d_vec);
bb = b_diag;
dd = d_vec;

% Прямой ход прогонки
for i = 2:n
    w = aa(i-1) / bb(i-1);
    bb(i) = bb(i) - w * cc(i-1);
    dd(i) = dd(i) - w * dd(i-1);
end

% Обратный ход прогонки
U = zeros(n,1);
U(n) = dd(n) / bb(n);
for i = n-1:-1:1
    U(i) = (dd(i) - cc(i) * U(i+1)) / bb(i);
end

% Оценка ошибки по Рунге
if ~first
    U_fine_on_coarse_nodes = U(1:2:end); % Берем каждую вторую точку
    err_est = max(abs(U_fine_on_coarse_nodes - U_prev)) / 3;
    fprintf('N = %d, h = %.6f, err_est = %.6e\n', N, h, err_est);

    if err_est < tol % Достигнута нужная точность
        x_final = x;
        U_final = U;
        U_prev_for_plot = U_prev;
        x_prev_for_plot = x_prev;
        break;
    end
else
    first = false; % Первый шаг - просто сохраняем решение
end

% Удваиваем число узлов для следующей итерации (сгущение сетки)
U_prev = U;
x_prev = x;
N = 2*N - 1;
end

fprintf('Runge error estimate: %.6e\n', err_est);

% График численного и точного решения
figure;
plot(x_final, U_final, 'ro-', 'MarkerSize', 8, 'LineWidth', 1.5);
hold on;
u_exact = @(x) x.^4 - 2*x.^3 + x.^2 + 3*x;
plot(x_final, u_exact(x_final), 'b-', 'LineWidth', 1.5);
hold off;
xlabel('x'); ylabel('u(x)');
legend('Numerical', 'Exact');
grid on;

% График оценки ошибки по Рунге
figure;
x_coarse = x_prev_for_plot;
U_coarse = U_prev_for_plot;

```

```

U_fine_at_coarse_nodes = U_final(1:2:end);
error_estimate = abs(U_fine_at_coarse_nodes - U_coarse) / 3;
plot(x_coarse, error_estimate, 'ko-', 'MarkerSize', 8, 'LineWidth', 1.5);
xlabel('x'); ylabel('Error estimate (Runge)');
grid on;

```

Приложение 3. Код программы для задачи Робина

```

graphics_toolkit("qt");

% Определяем границы интервала
a = 0; b = 1;

% Коэффициенты дифференциального уравнения
k = @(x) x.^2 + 1;
q = @(x) x + 1;
f = @(x) x.^5 - 21*x.^4 + 23*x.^3 - 14*x.^2 + 9*x - 2;

% Точное решение
u_exact = @(x) x.^4 - 2*x.^3 + x.^2 + 3*x;

% Параметры граничных условий третьего рода (Робена)
% Левая граница: alpha0*u(a) + alpha1*u'(a) = gamma_a
% Правая граница: beta0*u(b) + beta1*u'(b) = gamma_b
alpha0 = 1; alpha1 = 1; gamma_a = 3;
beta0 = 2; beta1 = 1; gamma_b = 9;

% Параметры метода
tol = 1e-4; % Требуемая точность
N = 5; % Начальное число узлов
err_est = tol + 1;
first = true;

% Основной цикл автоматического сгущения сетки
while err_est > tol
    h = (b - a)/(N - 1); % Шаг сетки
    x = linspace(a, b, N)'; % Узлы сетки
    xmid = (x(1:end-1) + x(2:end))/2; % Середины отрезков

    kmid = k(xmid);
    qv = q(x);
    fv = f(x);

    % Заполнение трехдиагональной матрицы
    a_diag = zeros(N,1); % Нижняя диагональ
    b_diag = zeros(N,1); % Главная диагональ
    c_diag = zeros(N,1); % Верхняя диагональ
    d_vec = zeros(N,1); % Вектор правой части

    % Внутренние узлы (конечно-разностная схема)
    for i = 2:N-1
        a_diag(i-1) = -kmid(i-1)/h^2;
        c_diag(i) = -kmid(i)/h^2;
        b_diag(i) = (kmid(i-1) + kmid(i))/h^2 + qv(i);
        d_vec(i) = fv(i);
    end

    % Левое граничное условие (Робена)
    k_minus_half = k(a - h/2);

```

```

k_plus_half = k(a + h/2);
b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1) - 2*k_minus_half*alpha0/(h*alpha1);
c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
d_vec(1) = fv(1) - 2*k_minus_half/(h*alpha1)*gamma_a;

% Правое граничное условие (Робена)
k_minus_half = k(b - h/2);
k_plus_half = k(b + h/2);
a_diag(N-1) = -(k_minus_half + k_plus_half)/h^2;
b_diag(N) = (k_minus_half + k_plus_half)/h^2 + qv(N) + 2*k_plus_half*beta0/(h*beta1);
d_vec(N) = fv(N) + 2*k_plus_half/(h*beta1)*gamma_b;

% Решение системы методом прогонки (Томаса)
aa = a_diag(1:N-1);
cc = c_diag(1:N-1);
n = length(d_vec);
bb = b_diag;
dd = d_vec;

% Прямой ход прогонки
for i = 2:n
    w = aa(i-1) / bb(i-1);
    bb(i) = bb(i) - w * cc(i-1);
    dd(i) = dd(i) - w * dd(i-1);
end

% Обратный ход прогонки (нахождение решения)
U = zeros(n,1);
U(n) = dd(n) / bb(n);
for i = n-1:-1:1
    U(i) = (dd(i) - cc(i) * U(i+1)) / bb(i);
end

% Оценка ошибки по Рунге (начиная со второго шага)
if ~first
    U_fine_on_coarse_nodes = U(1:2:end);
    err_est = max(abs(U_fine_on_coarse_nodes - U_prev)) / 3;
    fprintf('N = %d, h = %.6f, err_est = %.6e\n', N, h, err_est);

    if err_est < tol % Точность достигнута
        x_final = x;
        U_final = U;
        U_prev_for_plot = U_prev;
        x_prev_for_plot = x_prev;
        break;
    end
else
    first = false; % Первая итерация - только сохраняем решение
end

% Подготовка к следующей итерации (сгущение сетки в 2 раза)
U_prev = U;
x_prev = x;
N = 2*N - 1;
end

fprintf('Runge error estimate: %.6e\n', err_est);

% График 1: Численное и точное решение
figure;
plot(x_final, U_final, 'ro-', 'MarkerSize', 8, 'LineWidth', 1.5);

```

```

hold on;
plot(x_final, u_exact(x_final), 'b-', 'LineWidth', 1.5);
hold off;
xlabel('x'); ylabel('u(x)');
legend('Numerical solution', 'Exact solution', 'Location', 'northwest');
grid on;

% График 2: Оценка ошибки по Рунге
figure;
x_coarse = x_prev_for_plot;
U_coarse = U_prev_for_plot;
U_fine_at_coarse_nodes = U_final(1:2:end);
error_estimate = abs(U_fine_at_coarse_nodes - U_coarse) / 3;
plot(x_coarse, error_estimate, 'ko-', 'MarkerSize', 8, 'LineWidth', 1.5);
xlabel('x'); ylabel('Error estimate (Runge)');
grid on;

```

Приложение 4. Код программы для сравнения граничных условий Неймана

```

graphics_toolkit("qt");

% Определяем границы интервала
a = 0; b = 1;

% Коэффициенты
k = @(x) x.^2 + 1;
q = @(x) 10*(x + 1);
f = @(x) 10*x.^5 - 30*x.^4 + 14*x.^3 + 22*x.^2 + 36*x - 2;

% Параметры метода
tol = 1e-4;

% Определяем разные комбинации граничных условий
cases = [
    0, 0; % Случай 1: нулевые потоки
    2, 0; % Случай 2: поток 2 слева, 0 справа
    0, 2; % Случай 3: поток 2 справа, 0 слева
    2, 2 % Случай 4: поток 2 на обоих концах
];

% Создаем фигуру с 2x2 subplot для решений
figure('Name', 'Сравнение граничных условий Неймана');

for case_idx = 1:4
    gamma = cases(case_idx, 1);
    delta = cases(case_idx, 2);

    N = 5;
    err_est = tol + 1;
    first = true;

    while err_est > tol
        h = (b - a)/(N - 1);
        x = linspace(a, b, N)';
        xmid = (x(1:end-1) + x(2:end))/2;

        kmid = k(xmid);
    end
end

```

```

qv = q(x);
fv = f(x);

a_diag = zeros(N,1);
b_diag = zeros(N,1);
c_diag = zeros(N,1);
d_vec = zeros(N,1);

for i = 2:N-1
    a_diag(i-1) = -kmid(i-1)/h^2;
    c_diag(i) = -kmid(i)/h^2;
    b_diag(i) = (kmid(i-1) + kmid(i))/h^2 + qv(i);
    d_vec(i) = fv(i);
end

% Левое условие Неймана
k_minus_half = k(a - h/2);
k_plus_half = k(a + h/2);
b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1);
c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
d_vec(1) = fv(1) - 2*k_minus_half/h * gamma;

% Правое условие
k_minus_half = k(b - h/2);
k_plus_half = k(b + h/2);
a_diag(N-1) = -(k_minus_half + k_plus_half)/h^2;
b_diag(N) = (k_minus_half + k_plus_half)/h^2 + qv(N);
d_vec(N) = fv(N) + 2*k_plus_half/h * delta;

aa = a_diag(1:N-1);
cc = c_diag(1:N-1);
n = length(d_vec);
bb = b_diag;
dd = d_vec;

for i = 2:n
    w = aa(i-1) / bb(i-1);
    bb(i) = bb(i) - w * cc(i-1);
    dd(i) = dd(i) - w * dd(i-1);
end

U = zeros(n,1);
U(n) = dd(n) / bb(n);
for i = n-1:-1:1
    U(i) = (dd(i) - cc(i) * U(i+1)) / bb(i);
end

if ~first
    U_fine_on_coarse_nodes = U(1:2:end);
    err_est = max(abs(U_fine_on_coarse_nodes - U_prev)) / 3;

    if err_est < tol
        x_final = x;
        U_final = U;
        break;
    end
else
    first = false;
end

U_prev = U;

```



```

        x_prev = x;
        N = 2*N - 1;
    end

    % Подграфик
    subplot(2, 2, case_idx);
    plot(x_final, U_final, 'ro-', 'MarkerSize', 6, 'LineWidth', 1.2);
    hold on;
    u_exact = @(x) x.^4 - 2*x.^3 + x.^2 + 3*x;
    plot(x_final, u_exact(x_final), 'b-', 'LineWidth', 1.2);
    hold off;
    xlabel('x'); ylabel('u(x)');
    title(sprintf('gamma=%d, delta=%d', gamma, delta));
    grid on;
end

% Вторая фигура: графики погрешностей
figure('Name', 'Оценка погрешности');

for case_idx = 1:4
    gamma = cases(case_idx, 1);
    delta = cases(case_idx, 2);

    N = 5;
    err_est = tol + 1;
    first = true;

    while err_est > tol
        h = (b - a)/(N - 1);
        x = linspace(a, b, N)';
        xmid = (x(1:end-1) + x(2:end))/2;

        kmid = k(xmid);
        qv = q(x);
        fv = f(x);

        a_diag = zeros(N,1);
        b_diag = zeros(N,1);
        c_diag = zeros(N,1);
        d_vec = zeros(N,1);

        for i = 2:N-1
            a_diag(i-1) = -kmid(i-1)/h^2;
            c_diag(i) = -kmid(i)/h^2;
            b_diag(i) = (kmid(i-1) + kmid(i))/h^2 + qv(i);
            d_vec(i) = fv(i);
        end

        k_minus_half = k(a - h/2);
        k_plus_half = k(a + h/2);
        b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1);
        c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
        d_vec(1) = fv(1) - 2*k_minus_half/h * gamma;

        k_minus_half = k(b - h/2);
        k_plus_half = k(b + h/2);
        a_diag(N-1) = -(k_minus_half + k_plus_half)/h^2;
        b_diag(N) = (k_minus_half + k_plus_half)/h^2 + qv(N);
        d_vec(N) = fv(N) + 2*k_plus_half/h * delta;

        aa = a_diag(1:N-1);
    end
end

```

```

cc = c_diag(1:N-1);
n = length(d_vec);
bb = b_diag;
dd = d_vec;

for i = 2:n
    w = aa(i-1) / bb(i-1);
    bb(i) = bb(i) - w * cc(i-1);
    dd(i) = dd(i) - w * dd(i-1);
end

U = zeros(n,1);
U(n) = dd(n) / bb(n);
for i = n-1:-1:1
    U(i) = (dd(i) - cc(i) * U(i+1)) / bb(i);
end

if ~first
    U_fine_on_coarse_nodes = U(1:2:end);
    err_est = max(abs(U_fine_on_coarse_nodes - U_prev)) / 3;

    if err_est < tol
        x_final = x;
        U_final = U;
        U_prev_final = U_prev;
        x_prev_final = x_prev;
        break;
    end
else
    first = false;
end

U_prev = U;
x_prev = x;
N = 2*N - 1;
end

subplot(2, 2, case_idx);
x_coarse = x_prev_final;
U_coarse = U_prev_final;
U_fine_at_coarse_nodes = U_final(1:2:end);
error_estimate = abs(U_fine_at_coarse_nodes - U_coarse) / 3;
plot(x_coarse, error_estimate, 'ko-', 'MarkerSize', 6, 'LineWidth', 1.2);
xlabel('x'); ylabel('Error estimate');
title(sprintf('gamma=%d, delta=%d', gamma, delta));
grid on;
end

```

Приложение 5. Код программы для графика зависимости числа итераций от точности

```

graphics_toolkit("qt");

function [U_final, x_final, iter] = solve_dirichlet(a,b,k,q,f,u_exact,A,B,tol)
    N = 5;
    err_est = tol + 1;
    first = true;
    iter = 0;

```

```

while err_est > tol
    iter = iter + 1;
    h = (b - a)/(N - 1);
    x = linspace(a, b, N)';
    xmid = (x(1:end-1) + x(2:end))/2;
    kmid = k(xmid);
    qv = q(x);
    fv = f(x);
    a_diag = zeros(N,1);
    b_diag = zeros(N,1);
    c_diag = zeros(N,1);
    d_vec = zeros(N,1);
    for i = 2:N-1
        a_diag(i-1) = -kmid(i-1)/h^2;
        c_diag(i) = -kmid(i)/h^2;
        b_diag(i) = (kmid(i-1) + kmid(i))/h^2 + qv(i);
        d_vec(i) = fv(i);
    end
    b_diag(1) = 1; c_diag(1) = 0; d_vec(1) = A;
    a_diag(N-1) = 0; b_diag(N) = 1; d_vec(N) = B;
    n = length(d_vec);
    aa = a_diag(1:n-1);
    bb = b_diag;
    cc = c_diag(1:n-1);
    dd = d_vec;
    for i = 2:n
        w = aa(i-1) / bb(i-1);
        bb(i) = bb(i) - w * cc(i-1);
        dd(i) = dd(i) - w * dd(i-1);
    end
    U = zeros(n,1);
    U(n) = dd(n) / bb(n);
    for i = n-1:-1:1
        U(i) = (dd(i) - cc(i) * U(i+1)) / bb(i);
    end
    if ~first
        U_fine_on_coarse_nodes = U(1:2:end);
        err_est = max(abs(U_fine_on_coarse_nodes - U_prev)) / 3;
        if err_est < tol
            U_final = U;
            x_final = x;
            return;
        end
    else
        first = false;
    end
    U_prev = U;
    x_prev = x;
    N = 2*N - 1;
end
U_final = U;
x_final = x;
end

function [U_final, x_final, iter] = solve_neumann(a,b,k,q,f,gamma,delta,tol)
    N = 5;
    err_est = tol + 1;
    first = true;
    iter = 0;
    while err_est > tol
        iter = iter + 1;

```

```

h = (b - a)/(N - 1);
x = linspace(a, b, N)';
xmid = (x(1:end-1) + x(2:end))/2;
kmid = k(xmid);
qv = q(x);
fv = f(x);
a_diag = zeros(N,1);
b_diag = zeros(N,1);
c_diag = zeros(N,1);
d_vec = zeros(N,1);
for i = 2:N-1
    a_diag(i-1) = -kmid(i-1)/h^2;
    c_diag(i) = -kmid(i)/h^2;
    b_diag(i) = (kmid(i-1) + kmid(i))/h^2 + qv(i);
    d_vec(i) = fv(i);
end
k_minus_half = k(a - h/2);
k_plus_half = k(a + h/2);
b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1);
c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
d_vec(1) = fv(1) - 2*k_minus_half/h * gamma;
k_minus_half = k(b - h/2);
k_plus_half = k(b + h/2);
a_diag(N-1) = -(k_minus_half + k_plus_half)/h^2;
b_diag(N) = (k_minus_half + k_plus_half)/h^2 + qv(N);
d_vec(N) = fv(N) + 2*k_plus_half/h * delta;
aa = a_diag(1:N-1);
cc = c_diag(1:N-1);
n = length(d_vec);
bb = b_diag;
dd = d_vec;
for i = 2:n
    w = aa(i-1) / bb(i-1);
    bb(i) = bb(i) - w * cc(i-1);
    dd(i) = dd(i) - w * dd(i-1);
end
U = zeros(n,1);
U(n) = dd(n) / bb(n);
for i = n-1:-1:1
    U(i) = (dd(i) - cc(i) * U(i+1)) / bb(i);
end
if ~first
    U_fine_on_coarse_nodes = U(1:2:end);
    err_est = max(abs(U_fine_on_coarse_nodes - U_prev)) / 3;
    if err_est < tol
        U_final = U;
        x_final = x;
        return;
    end
else
    first = false;
end
U_prev = U;
x_prev = x;
N = 2*N - 1;
end
U_final = U;
x_final = x;
end
function [U_final, x_final, iter] = solve_robin(a,b,k,q,f,alpha0,alpha1,gamma_a,beta0,beta1,gamma_b,tol)

```

```

N = 5;
err_est = tol + 1;
first = true;
iter = 0;
while err_est > tol
    iter = iter + 1;
    h = (b - a)/(N - 1);
    x = linspace(a, b, N)';
    xmid = (x(1:end-1) + x(2:end))/2;
    kmid = k(xmid);
    qv = q(x);
    fv = f(x);
    a_diag = zeros(N,1);
    b_diag = zeros(N,1);
    c_diag = zeros(N,1);
    d_vec = zeros(N,1);
    for i = 2:N-1
        a_diag(i-1) = -kmid(i-1)/h^2;
        c_diag(i) = -kmid(i)/h^2;
        b_diag(i) = (kmid(i-1) + kmid(i))/h^2 + qv(i);
        d_vec(i) = fv(i);
    end
    k_minus_half = k(a - h/2);
    k_plus_half = k(a + h/2);
    b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1) - 2*k_minus_half*alpha0/(h*alpha1);
    c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
    d_vec(1) = fv(1) - 2*k_minus_half/(h*alpha1)*gamma_a;
    k_minus_half = k(b - h/2);
    k_plus_half = k(b + h/2);
    a_diag(N-1) = -(k_minus_half + k_plus_half)/h^2;
    b_diag(N) = (k_minus_half + k_plus_half)/h^2 + qv(N) + 2*k_plus_half*beta0/(h*beta1);
    d_vec(N) = fv(N) + 2*k_plus_half/(h*beta1)*gamma_b;
    aa = a_diag(1:N-1);
    cc = c_diag(1:N-1);
    n = length(d_vec);
    bb = b_diag;
    dd = d_vec;
    for i = 2:n
        w = aa(i-1) / bb(i-1);
        bb(i) = bb(i) - w * cc(i-1);
        dd(i) = dd(i) - w * dd(i-1);
    end
    U = zeros(n,1);
    U(n) = dd(n) / bb(n);
    for i = n-1:-1:1
        U(i) = (dd(i) - cc(i) * U(i+1)) / bb(i);
    end
    if ~first
        U_fine_on_coarse_nodes = U(1:2:end);
        err_est = max(abs(U_fine_on_coarse_nodes - U_prev)) / 3;
        if err_est < tol
            U_final = U;
            x_final = x;
            return;
        end
    else
        first = false;
    end
    U_prev = U;
    x_prev = x;
    N = 2*N - 1;
end

```

```

    end
    U_final = U;
    x_final = x;
end

% Параметры задач (менять здесь)
a = 0; b = 1;

% Дирихле
k_D = @(x) x.^2 + 1;
q_D = @(x) x + 1;
f_D = @(x) x.^5 - 21*x.^4 + 23*x.^3 - 14*x.^2 + 9*x - 2;
u_exact_D = @(x) x.^4 - 2*x.^3 + x.^2 + 3*x;
A = u_exact_D(a); B = u_exact_D(b);

% Неймана
k_N = @(x) x.^2 + 1;
q_N = @(x) 10*(x + 1);
f_N = @(x) 10*x.^5 - 30*x.^4 + 14*x.^3 + 22*x.^2 + 36*x - 2;
gamma = 3; delta = 3;

% Робина
k_R = @(x) x.^2 + 1;
q_R = @(x) x + 1;
f_R = @(x) x.^5 - 21*x.^4 + 23*x.^3 - 14*x.^2 + 9*x - 2;
alpha0 = 1; alpha1 = 1; gamma_a = 3;
beta0 = 2; beta1 = 1; gamma_b = 9;

tol_list = [1e-3, 1e-4, 1e-5, 1e-6, 1e-7];
iter_D = zeros(length(tol_list), 1);
iter_N = zeros(length(tol_list), 1);
iter_R = zeros(length(tol_list), 1);

for i = 1:length(tol_list)
    [~, ~, iter_D(i)] = solve_dirichlet(a,b,k_D,q_D,f_D,u_exact_D,A,B,tol_list(i));
    [~, ~, iter_N(i)] = solve_neumann(a,b,k_N,q_N,f_N,gamma,delta,tol_list(i));
    [~, ~, iter_R(i)] = solve_robin(a,b,k_R,q_R,f_R,alpha0,alpha1,gamma_a,beta0,beta1,gamma_b,tol_list(i));
end

figure;
subplot(1,3,1);
semilogx(tol_list, iter_D, 'ro-', 'MarkerSize', 8, 'LineWidth', 1.5);
set(gca, 'XDir', 'reverse');
xlabel('Точность');
ylabel('Количество итераций');
title('Дирихле');
grid on;
set(gca, 'XMinorGrid', 'off');

subplot(1,3,2);
semilogx(tol_list, iter_N, 'bo-', 'MarkerSize', 8, 'LineWidth', 1.5);
set(gca, 'XDir', 'reverse');
xlabel('Точность');
ylabel('Количество итераций');
title('Неймана');
grid on;
set(gca, 'XMinorGrid', 'off');

subplot(1,3,3);
semilogx(tol_list, iter_R, 'go-', 'MarkerSize', 8, 'LineWidth', 1.5);
set(gca, 'XDir', 'reverse');

```

```
xlabel('Точность');  
ylabel('Количество итераций');  
title('Робина');  
grid on;  
set(gca, 'XMinorGrid', 'off');
```